

Object-Oriented Programming

05

Hi! I recently watched a digital encyclopedia and I'm inspired to make my own. Can you help me get started?

Absolutely! That's a fantastic project to work on. We can create a software application for a digital Encyclopedia with features like 3D models and user interactivity. We can implement it using Object-Oriented Programming. Let's get started.



Books have always been a great knowledge-gaining source. Dictionaries came into existence to understand the words or the things that we don't know. This resulted in the creation of an Encyclopedia. An Encyclopedia is a comprehensive reference book with detailed information for gaining knowledge of various things all around the world. It usually consists of high-resolution images to support detailed information.



Image Credits: Britannica.com

Nowadays, we can have all such information on computers via applications and websites. Websites keep the audience engaged by using high-resolution images, videos, audios, and interactive elements.

For example, we can explore the National Geographic Kids Website, which is full of Images and videos. - <https://kids.nationalgeographic.com/>

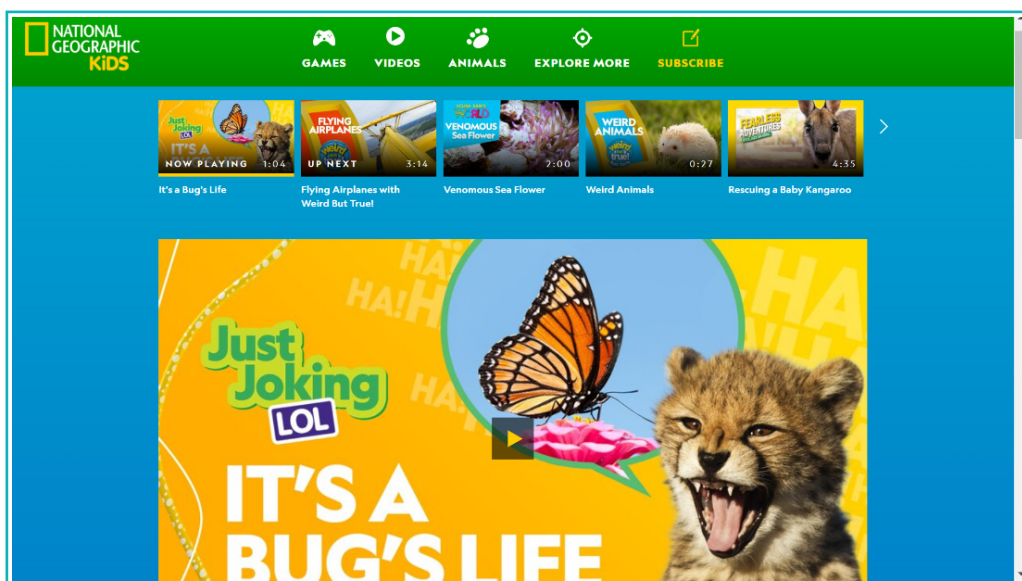
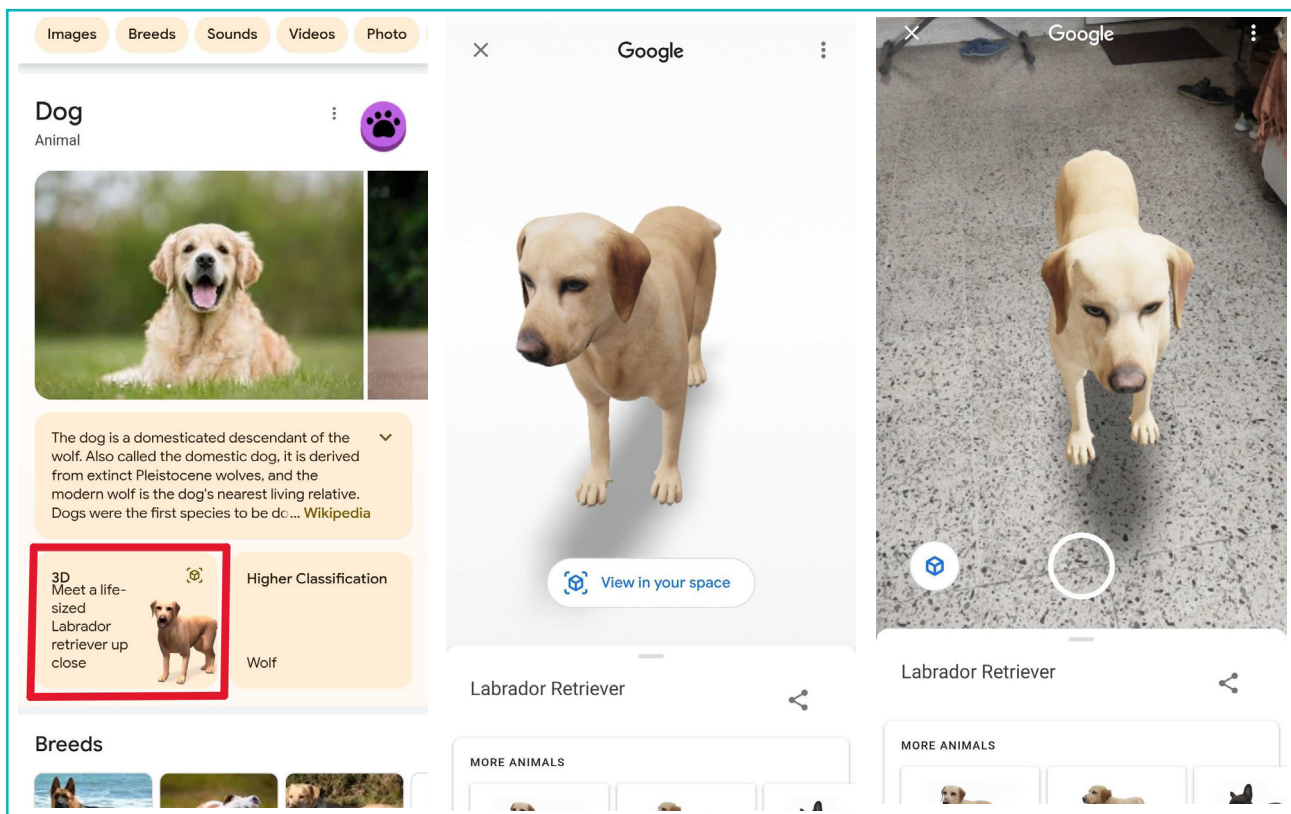


Image Credits: National Geographic Society

Modern websites also exhibit 3D models and animations, along with 2D images, for a better understanding of concepts and a good user. Interactive elements like 3D objects, that users can move or rotate to get a 360-degree detailed perspective, enhance user experience.

For example, if we search for 'dog' or any animal in the Google search on a smartphone, we can see an option - 'View in 3D'. When we click on the button, we will see a 3D model of a dog in augmented reality that we can zoom in, zoom out, or rotate to observe from any angle.



Activity

Similarly, another example is 'Google Earth' where we can see the Earth in 3D, and rotate or zoom it using the mouse to see the different parts of the Earth.

Check the 3D model of the Earth on the given website: <https://earth.google.com/>



In this lesson, we will create a web platform using p5js that will display planets in our solar system. We will use a 3D platform and also include interactive elements to enhance the user experience. A rough User Interface can be imagined as follows:



Activity

Programming and Real World

Programming applications help us in a variety of ways, in our daily lives to make life easier. Hence, programming languages are inspired by the design of the world around us. For example, we have objects everywhere around us with names and properties; each object may perform certain tasks, and so on. Similarly, we can create data entities to store data called 'objects' that can have properties and functionalities. This approach in coding is called **Object-Oriented Programming (OOP)**.

As we move forward, we develop more complicated applications. Here, Object-Oriented Programming comes into the picture, which helps in the processing of complex data.

In this lesson, we will introduce OOP and its fundamental element 'Objects'. Let us start with an activity by creating a graphical representation of a planet in p5js.

Exercise

■ Answer the question with one sentence.

1 What is Object-Oriented Programming?

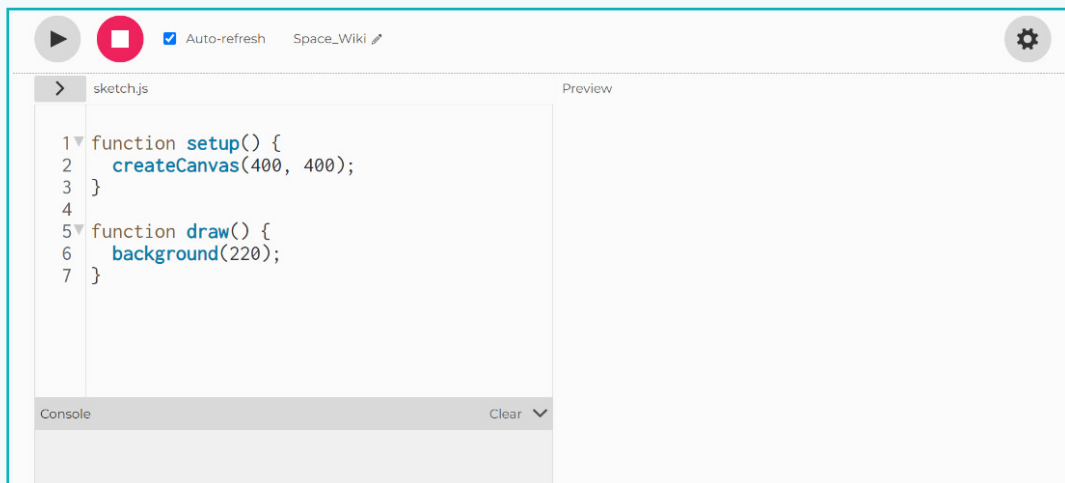


Activity

One of the planets nearest to the Earth is 'Mars', also known as the 'Red Planet'. It is mostly reddish orange in colour. Let us start by drawing a red-coloured circle on the canvas.

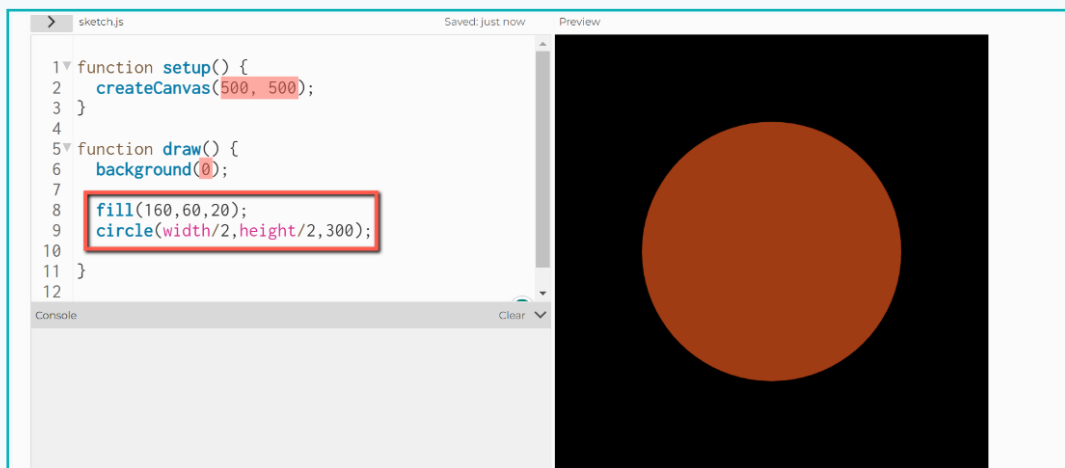
Create the Planet

- 1 Open the p5js editor.



2 Draw mars

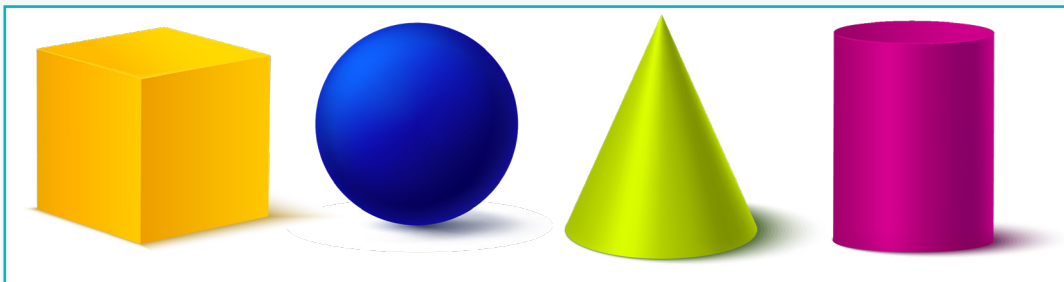
- Change the 'canvas size' to '500x500 pixels' and the 'background colour' to 'black'.
- Draw a circle at the centre of the canvas of '300 pixels' diameter and a colour like that of Mars i.e., dark orange.



Activity

We can see a dark orange coloured circle that represents Mars. To make it more realistic, instead of 2D, we can add a 3D object to display the Mars planet.

p5.js is a powerful tool for creating 2D graphics and by default, canvas is in the 'P2D' mode for rendering in 2D. It also offers the ability of rendering 3D graphics using the 'WEBGL' library, which enables the third dimension, 'Z', within a web browser.



Creating 3D designs can be difficult since we need to add features to it such as shape, texture, and lighting to the 3D models. Computers contain a particular element called the **Graphics Processing Unit** (GPU) that handles the complex calculations required to render such 3D models. The GPU can operate on multiple tasks at once when we are dealing with pixels and various shapes in 3D space.



WEBGL library provides a 3D rendering technique in p5js that allows us to display 3D shapes and objects with features such as lighting, texture, and viewing camera angle.

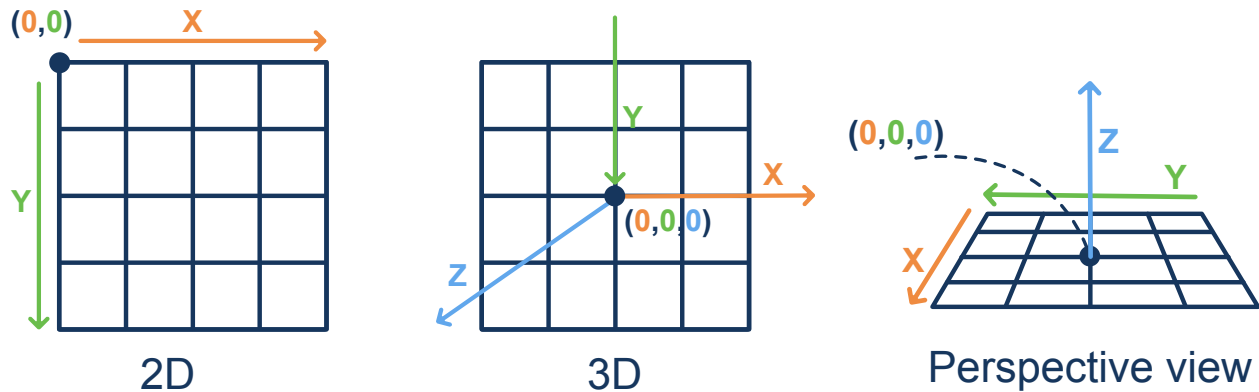
We can enable the 3D mode by adding the keyword 'WEBGL' in the 'createCanvas()' command as the third parameter.

Syntax: `createCanvas(width, height, WEBGL);`

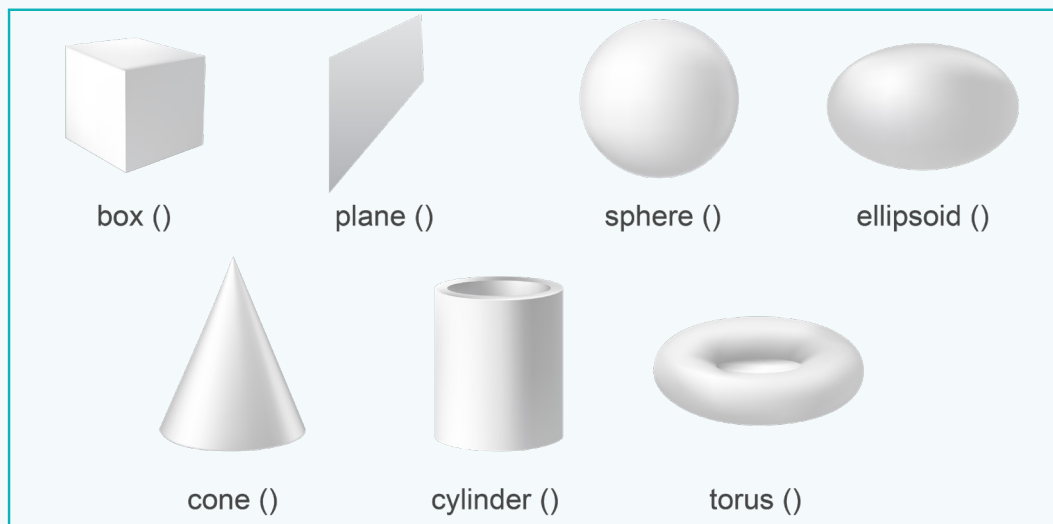
Activity

Note:

When drawing in 2D, the point (0,0) is located at the top left corner of the screen. However, in the 'WebGL' mode, the origin of the sketch (0,0,0) is in the centre of the screen. By default, the x-axis moves left to right, the y-axis moves up to down, and the z-axis moves from further to closer.



Now, with these fundamentals in place, we can add 3D objects. There are many basic shapes that we can draw using simple commands in p5js.



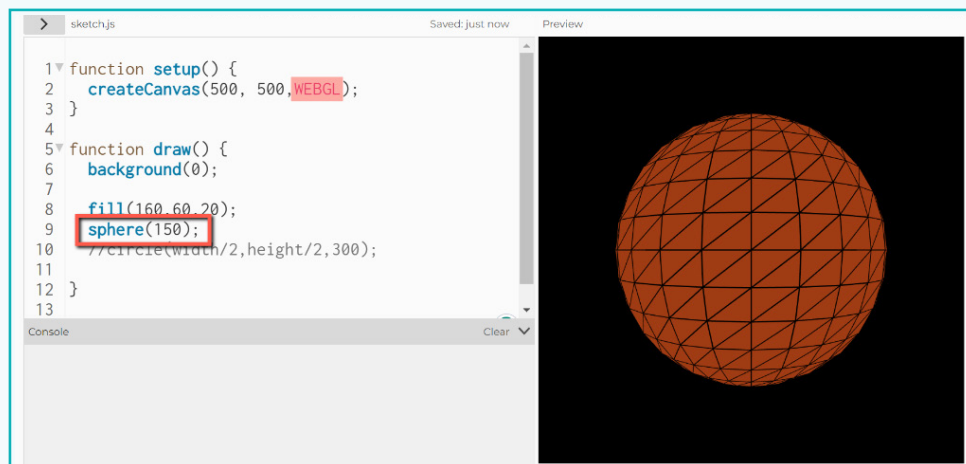
Activity

For adding a planet, we can use a 'sphere' shape.

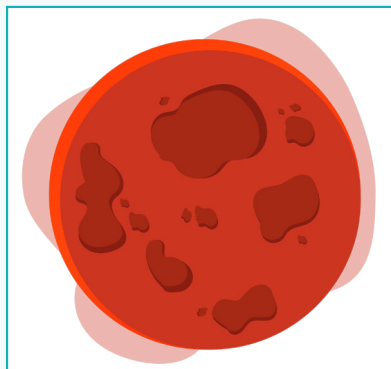
sphere() – This command draws a sphere with the given radius.

3 Add a sphere instead of a circle

- Add the keyword 'WEBGL' in the 'createCanvas()' method as a third parameter. Add a comma before the 'WEBGL' keyword.
- Comment out the 'circle()' command and replace it with the 'sphere()' command with a '150 pixels' radius as an input parameter.



We can see a sphere with some geometrical lines. We can make it look realistic by removing the geometrical lines and using hues that resemble the surface aesthetics of Mars. For example, the tint of the sphere is quite similar to the colour of the Mar's surface. We can also add a few shapes and patches to make it look more like Mars. However, visuals of actual Mars's surface include craters and patterns with incredibly complicated hues of varied colours.

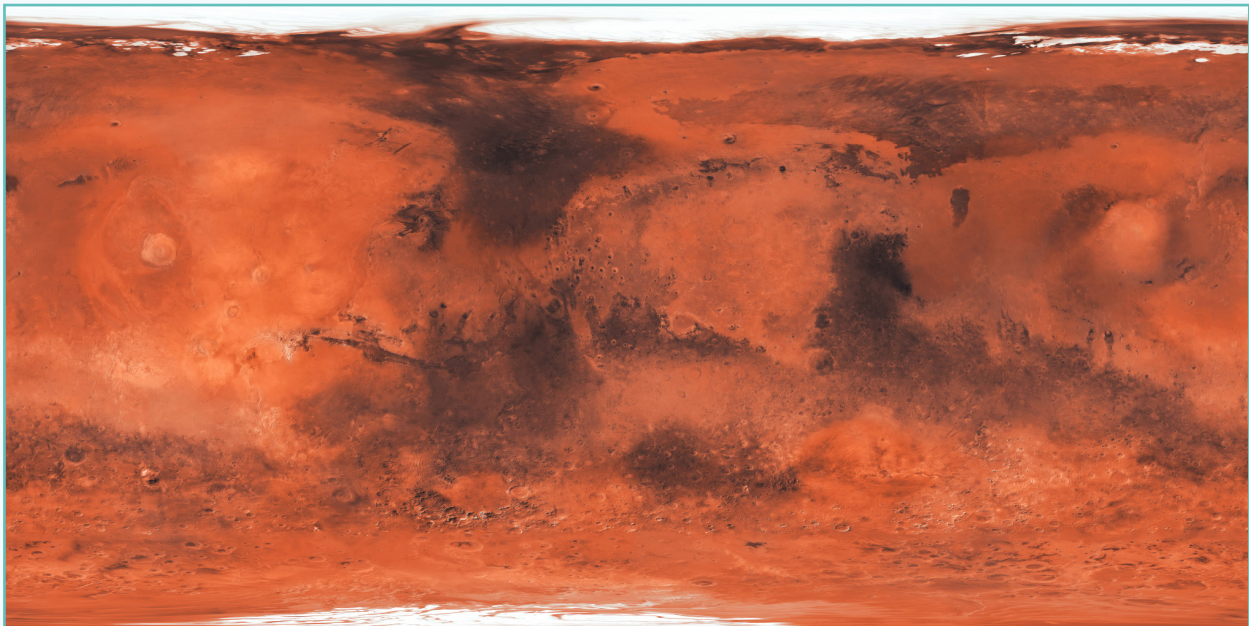


Activity

We can make it look very realistic by adding texture instead of applying colours.

Texture refers to the visual and tactile characteristics of the surface. It is similar to a 'skin' that wraps around a 3D geometry. Even though the surface details are not there in 3 dimensions, a texture makes it appear to have them. For example, uneven surfaces or surface colors on the earth that we can see in google earth. Images and video are the only textures that are currently supported.

We can use digital images of texture to replicate the appearance of different materials, such as wood, metal, fabric, or any surface. The complex visual properties applied through textures give detailed and realistic look to the 3D models. In this case, we can use the following image of the texture of the Mars surface.



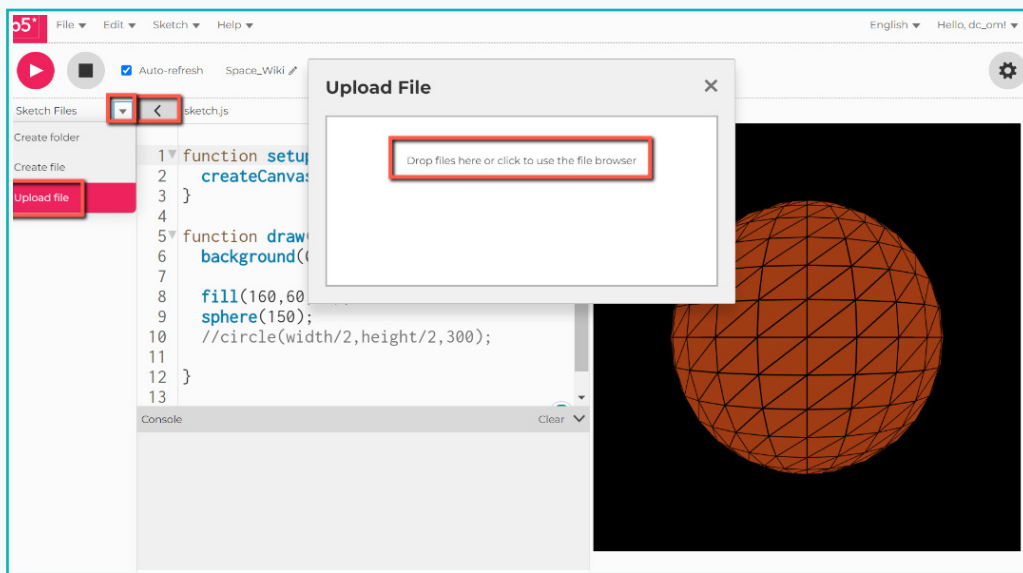
We can download this image from a folder with the texture of various planets using the following QR Code.



Activity

4 Add Texture

- a. Upload the texture image provided in the resources to the 'Sketch Files' section.



We will be able to see the image 'mars.jpg' in 'Sketch Files'. To remove the geometrical lines and apply texture on the surface, we can load the image in the code and use the 'texture()' and 'normalMaterial()' commands respectively.

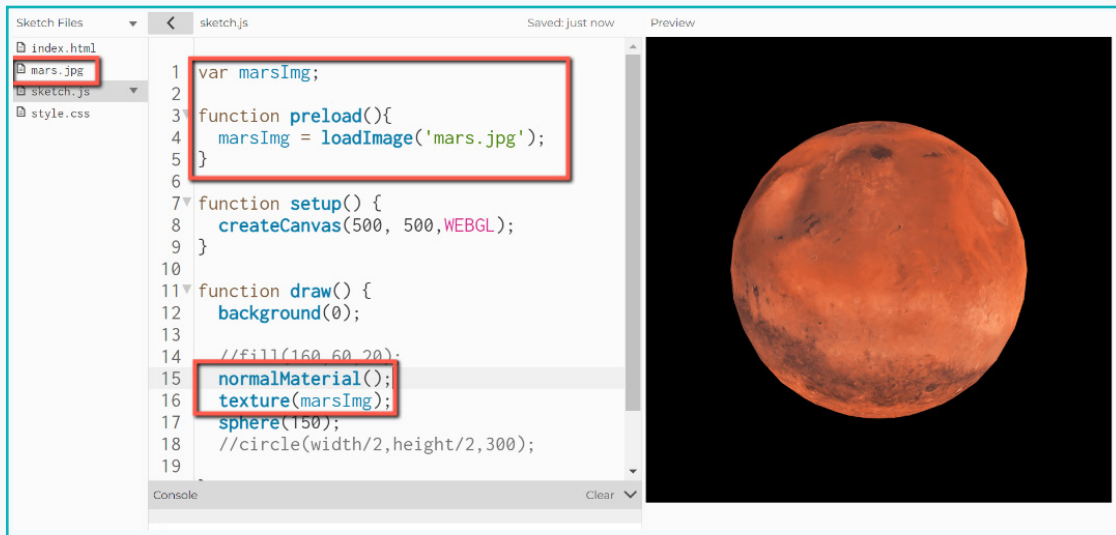
normalMaterial(): This command sets the current material as a normal material and removes geometrical lines. It is not affected by light.

texture(): This command sets the input image or video as the texture to render subsequent shapes.

- b. Add a preload function that will load the image into a variable at the beginning of code execution.
- c. Add 'loadImage()' command in the 'preload' function with the path to the image as an input and assign it to the variable 'marsImg' to save the image in a variable.
- d. Add the 'normalMaterial()' command before the 'sphere()' command to remove the geometrical lines from the sphere in draw().

Activity

- e. Add the 'texture()' command and add an image 'marsImg' as an input to add texture to the sphere after normalMaterial() command.



Now we can see that the texture image is wrapped over the sphere to make it look realistic.

Exercise

Tick mark ☒ the correct answer

- 2 What is the right order of execution of the following functions?

A) setup() B) draw() C) preload().

- | | | | |
|----------|--------------------------|----------|--------------------------|
| a. A-B-C | ☐ | b. B-C-A | ☐ |
| c. C-A-B | ☐ | d. A-C-B | ☐ |

- 3 The _____ command adds texture to the surface of a 3D object.

- | | | | |
|--------------|--------------------------|---------------------|--------------------------|
| a. preload() | ☐ | b. loadImage() | ☐ |
| c. texture() | ☐ | d. normalMaterial() | ☐ |

Exercise

 Tick mark ☒ the correct answer

4 The _____ mode is used to render objects in 3D in p5js.

a. CENTER ☐

b. WEBGL ☐

c. DEGREES ☐

d. None ☐

Activity

We will move the sphere upwards to make room underneath to show planet information later. We can use `translate()` for the same purpose.

`translate()` shifts the origin by given dimensions. It accepts arguments for (x,y) or (x,y,z) values to apply the translation along each axis. Everything we draw after calling `translate()` will be positioned relative to that point.

We can also add rotation animation to the sphere object so that the user can see all parts of the planet while it rotates around its own axis. To rotate the sphere along the y-axis, we can use the '`rotateY()`' command before the '`sphere()`' command. However, we need to give a constant linearly increasing number as an angle input to the '`rotate`' command. We can either create a counter or use the '`millis()`' method.

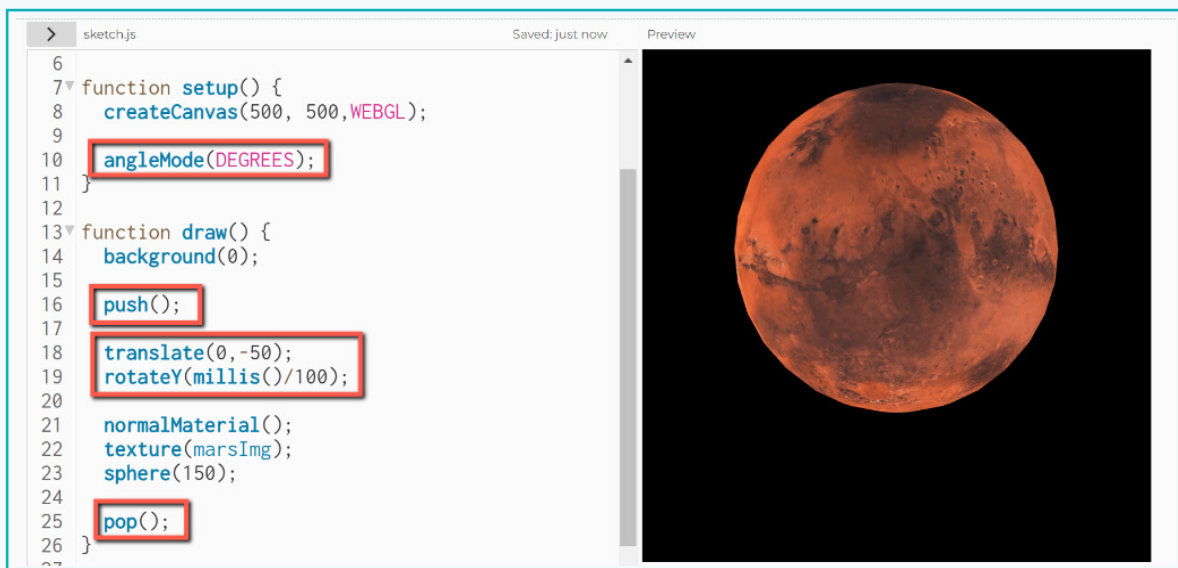
Millis(): The '`Millis`' method returns the number of milliseconds (one thousandth of a second) since starting the code execution, i.e., when `setup()` was called.

We can use the '`millis()`' command as an input to the '`rotateY`' command. However, the output value of the '`millis()`' increases rapidly. This can be checked using a console. Therefore, we can reduce the speed by dividing '`millis()`' by '100' and using it as an input to the '`rotateY()`' command.

Activity

5 Add rotation to 3D object

- Use the 'rotateY()' method with 'millis()/100' as an input. Add the 'angleMode(DEGREES)' command in setup to consider the input values in degrees instead of radians.
- Add the 'translate(0,-50)' command before rotate() to move the sphere upwards.
- Encapsulate code related to a sphere using the 'push()' and 'pop()' methods to apply changes exclusively to the Mars sphere.



We can see that Mars is rotating around its own axis. This looks realistic and engaging compared to any 2D image.

Activity

Display Information

Now that the 3D model is ready, we can display planet's information, such as name, diameter, number of moons, distance from the sun, gravitational force, etc. The following chart depicts various properties of planets in our solar system

Planet Name	Diameter (km)	Number of moons	Gravity (m/s ²)	Length of a day
Mercury	4,879	0	3.7	59 days
Venus	12,104	0	8.9	243 days
Earth	12,760	1	9.8	24 hours
Mars	6,792	2	3.7	24 hrs 37 min
Jupiter	1,42,984	92	23.1	9 hr 55 min
Saturn	1,20,536	146	9.0	10 hr 40 min
Uranus	51,118	27	8.7	16 hr 50 min
Neptune	49,528	14	11.0	6 hr 11 min

Note:

These values are subject to change, though taken from an authentic source from the NASA website, as continuous research results in changes.

For simplicity, we will add only two properties— 'name' and 'diameter'. We will use variables to save this data.

Activity

1 Create variables for displaying information

- Create new variables 'name' and 'dia' to store the name and the diameter of the planet, respectively.
- Initialize the variables in setup() as: 'name' as "Mars" and 'dia' as "6,792 km".



```
> sketch.js Saved: just now
1  var marsImg;
2  var name;
3  var dia;
4
5  function preload(){
6    marsImg = loadImage('mars.jpg');
7  }
8
9  function setup() {
10   createCanvas(500, 500, WEBGL);
11   angleMode(DEGREES);
12
13   name = "Mars";
14   dia = "6,792 km";
15 }
16
Console Clear
```

Note:

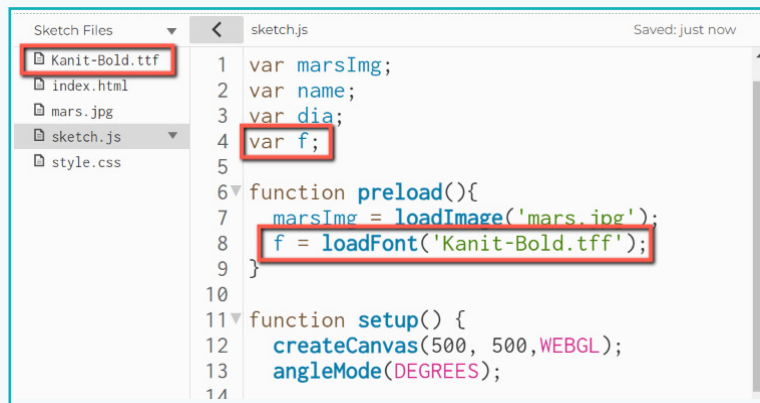
The value inside the 'dia' variable is a string and not a 'number' datatype.

In p5.js, we use the 'text()' command to display text on the screen. We can customize the font and size of the text. By default, p5.js uses a standard font for the text. However, when using the WebGL mode, we can only use fonts that have been loaded using the 'loadFont()' method. This means we need to upload a font file in the '.tff' format and include it in our code. In this session, we will use a font called 'Kanit-Bold', which can be found in the resources folder shared through a QR code.

Activity

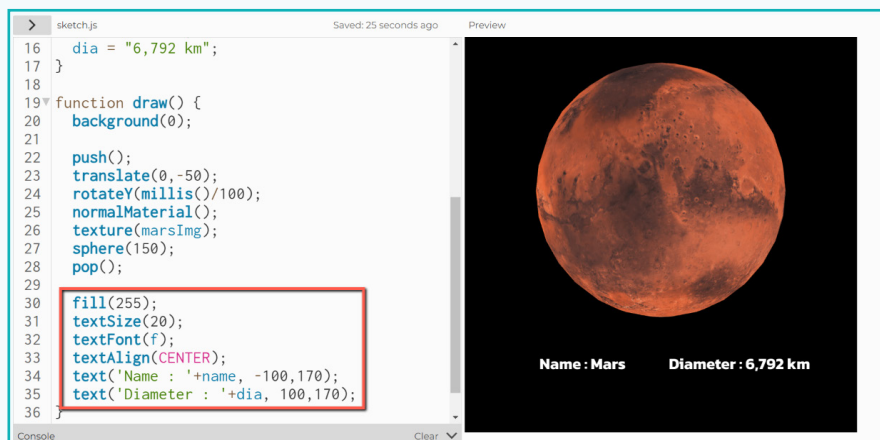
2 Add information text

- Download the 'Kanit-Bold.ttf' file from the resources folder. Upload the file in the p5js editor sketch files.
- Declare a variable 'f' at the beginning. Use the 'loadFont()' command to load the 'Kanit-Bold.ttf' font and assign it to the 'f' variable in the 'preload()' function.



Now, we will start adding the code to display text after the code is added to display Mars in 3D.

- Add the 'fill(255)' command to fill the text with white colour. Note that when text is added, it is considered as a shape that has a border and a fill area inside.
- Use the 'textSize()' command and set it to '20'. Try changing the size later when the text is displayed.
- Use the 'textFont()' command and set it to the 'f' variable to have the 'Kanit-Bold' font. Use the 'textAlign(CENTER)' command to align text centrally.
- Use the 'text()' command to display the name and diameter in the format 'property_Name : property_Value' just below the sphere.



Activity

Now, we can observe Mars with realistic texture in 3D and its properties. We can also display more properties or information, if required. This approach to learning about planets is much more engaging and informative than conventional methods.

Exercise

■ State if the statements are True or False

- 5 We must load a font in the 'WebGL' mode to be able to display the text, although the default font can be used in 2D mode.

▲ _____

- 6 The 'Millis()' command provides the number of milliseconds passed since the execution of the code is started

▲ _____

- 7 Translate() command is used to set or change the position of the sphere shape on the canvas.

▲ _____

We have been using variables to save and display different information about planets. However, if we want to show more details, we will need even more variables, which can make the code long and difficult to manage. Plus, if we want to show information about many planets, we will need even more variables. This is when arrays can be helpful. Arrays allow us to organise information together. For instance, if we want to display details about planets such as Mars, Earth, and Jupiter, we can create arrays for their names, diameters, and any other information we want to show.

```
Var names = ["Mars","Earth","Jupiter"];
```

```
var dia = ["6792 km","12,760 km","1,42,984 km"];
```

and more...

When it comes to multiple properties of the planets, we will have to use multiple arrays. We must create separate arrays to display different properties. Also, each array must have the same order of elements as the sequence of planets. All of this complicates the code. However, we can make things easier by using Object-Oriented Programming. This approach both simplifies the code and makes it more organised.

What is OOP?

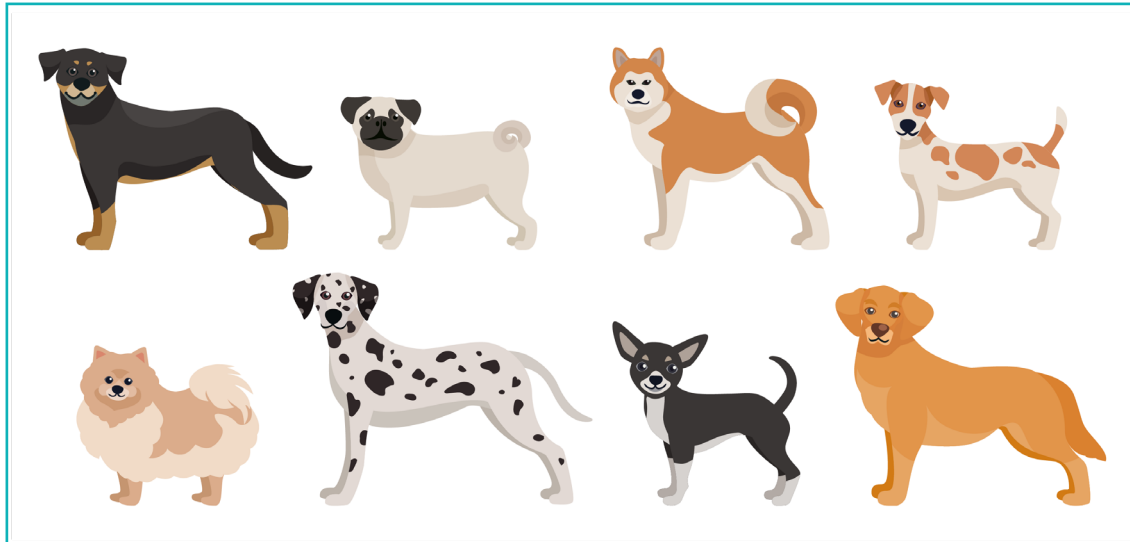
In real life, we interact with various objects around us. For example, we eat food on a plate with a spoon. In this process, food, plate, spoon, and the person eating, all interact with each other in a certain way. Each object has its own properties and functions, such as the plate, which is usually flat and can contain food. The spoon, which can be held with a hand, is used to pick up a bite and put it in the mouth.



Similarly, the entire world around us comprises objects that interact with ourselves and with one another. As programming is mainly used to create real-world applications, the same concept is used in programming called 'Object-Oriented Programming'. This concept simplifies the development of real-world applications.

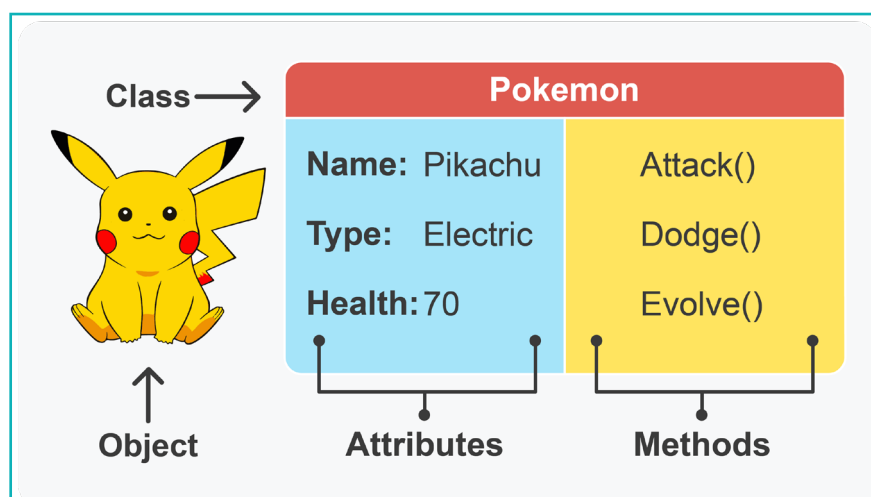
Generally, objects having similar properties are classified in a group or class. For example, 'animal' is a class with several elements such as human, cat, dog, tiger, elephant, etc.

Consider another example of the dog as a class. Its objects can be different dogs with properties such as name, breed, colour, and functions like walk, bark, run, etc. Similarly, in OOPs, we can create a class with common properties and methods. Multiple objects can be created from a class that can have common property types.



Designed by Freepik

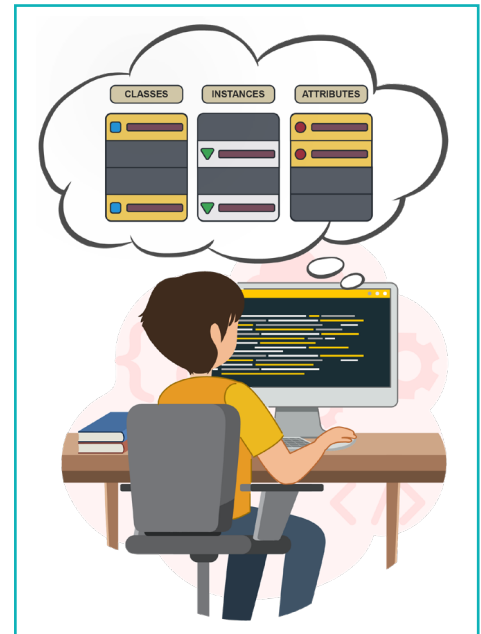
Let us look at another example. If we are creating a game that has Pokémon, there will be several Pokémon with different properties and actions. We can create a class 'Pokémon'. We can create multiple objects from the class 'Pokémon' like Pikachu, and associate them with all the property variables and functionality methods of the class with the object.



In Object-Oriented Programming, multiple variables of different data types and methods are associated with a single object. This makes programming much simpler and code easy to manage.

Object-Oriented Programming (OOP) – This is a programming paradigm that organizes code based on the concept of objects, each of which can have its own data (attributes) and behaviours (methods). We can use those objects in programs which can interact with users as well as among themselves to build complex software systems. This approach in programming is known as ‘Object-Oriented Programming’.

Example: We are creating a platform that will display planets and their information. The code we have is a procedural structure with variables for properties such as name and diameter. However, if we want to show multiple planets, we can create objects for planets that store the properties and methods of each planet.



We will learn more about the class in the later chapter. In this session, we will only introduce what objects are and how to use them.

Object – An **Object** is a unique data entity that can contain the properties and methods. Objects can be created independently, similar to defining a variable or they can be created from a class. If the object is created using a class, then it is termed as ‘instance of a class’.

For example, a ‘car’ is a real-life object, which has characteristics like colour, type, model, horsepower etc., and performs certain actions like motion. The characteristics of an Object are called ‘Properties’ and the actions are called ‘Methods in Object-Oriented Programming’.

What properties can we identify of the planet in the code we created earlier? We can review the code and identify properties such as– name, diameter, and path of the texture image associated with the planet object.

Object Literal: Objects can be created independently using ‘Object Literal’ in JavaScript. ‘Object Literal’ is a comma-separated list of name-value pairs inside curly braces. Therefore the syntax to create objects has curly brackets {...} with a list of properties separated by commas. A property is a ‘key: value’ pair, where a key is a property name, and the value can be anything. Following are the syntax rules:

- A colon separates the property name from the value.
- A comma separates each name-value pair from the next.
- A comma after the last name-value pair is optional.

```
var myObject = {  
    myProperty: value,  
    secondProperty: value,  
    myMethod: function () {  
        //code here  
    },  
    secondMethod: function () {  
        //more code  
    }  
}
```

If any of the syntax rules are broken, such as a missing comma or colon or curly bracket, an error will be generated. Note that the Objects can also have methods. Methods are stored in properties as function definitions.

For example, if we want to create an object for mars planet, we can have property variables such as name, diameter, and image path. We can create an object ‘Mars’ as follows:

```
var mars = {  
    imgPath : “mars.jpg”,  
    name : “Mars”,  
    dia : “6,787 km”,  
};
```

The individual property of an object can be accessed using the object name and key joined with a ‘dot (.)’ operator.

For example, to access the value of the ‘imgPath’ property of the object ‘mars’, we can write

‘mars.imgPath’.

Use the console with the 'console.log(mars.imgPath)' command and check the output. Similarly, we can check values in other properties.

Question: Can you create an object for the Earth based on the 'Mars' object we have created so far using the property variables we have used?

Ans:

```
var earth = {  
  imgPath : "earth.jpg",  
  name : "Earth",  
  dia : "12,760 km",  
};
```

Activity

Now, let us implement the concept of an object into code.

3 Create mars object

- Comment out or remove the variable declaration and initialization in the setup.
- Create an object 'Mars' with properties 'imgPath', 'name', and 'dia' at the beginning of the code.



```
> sketch.js Saved: just now  
1 // var marsImg;  
2 // var name;  
3 // var dia;  
4  
5 var mars = {  
6   imgPath: "mars.jpg",  
7   name: "Mars",  
8   dia: "6,787 km",  
9 }  
10 var t;  
11  
12 function preload(){  
13   marsImg = loadImage(mars.imgPath);  
14   f = loadFont('Kanit-Bold.ttf');  
15 }  
16  
17 function setup() {  
18   createCanvas(500, 500, WEBGL);  
19   angleMode(DEGREES);  
20   // name = "Mars";
```

Activity

Note that we had created the 'marsImg' variable to store the texture image of the mars. In the object, we did not create a property to store the image. Instead, we added a 'String' property to store the image path.

We can also add a new object property or update the existing one directly using the 'dot (.)' notation. We can create the new object property as 'mars.img' instead of the 'marsImg' variable.

4 Add image object property

- Replace the 'marsImg' variable by creating the new object property 'mars.img' in the 'preload' function.
- Use the object and its property 'mars.imgPath' instead of the 'imgPath' variable for loadImage() command in the 'preload' function.
- Use the console to check the contents and the new 'img' property in the 'mars' object using the 'console.log()' command.



```
> sketch.js Saved: just now
1 var mars = {
2   imgPath: "mars.jpg",
3   name: "Mars",
4   dia: "6,787 km",
5 }
6
7 var f;
8
9 function preload(){
10   mars.img = loadImage(mars.imgPath);
11   f = loadFont('Kanit-Bold.ttf');
12   console.log(mars);
13 }
14
```

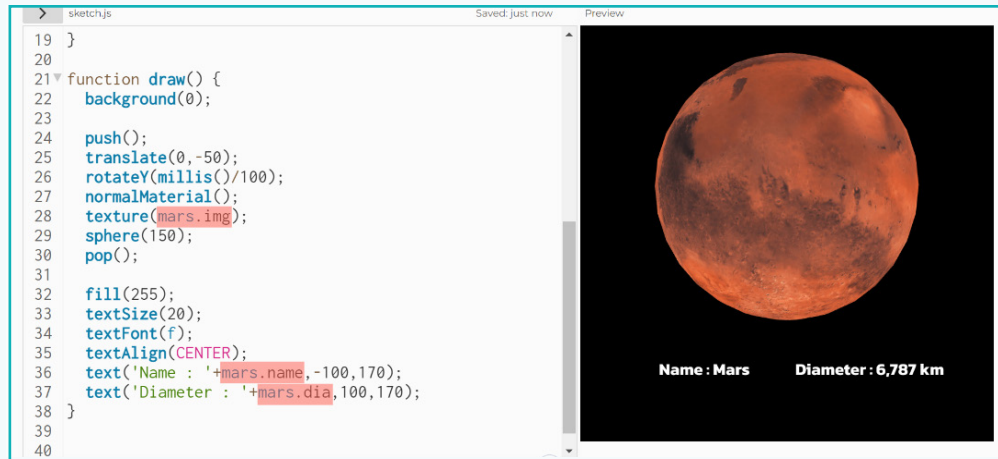
Console Clear

```
{imgPath: "https://assets.editor.p5js.org/5f7315c76d0c9e0024ff08bc/232966de-ae91-4fb7-89d5-fe0712d62ce4.jpg", name: "Mars", dia: "6,787 km", img: Object}
```

5 Update the code with object properties

- Update the 'marsImg' variable in the rest of the code with the 'mars.img' object property.
- Also, update the variables 'name' and 'dia' with the object properties 'mars.name' and 'mars.dia' in the text command, respectively

Activity



Now we can see the same result as earlier, with a rotating Mars sphere and basic information displayed at the bottom. We can also add and display more properties from the table.

Exercise

Fill in the Blanks.

- 8 _____ is a comma-separated list of name-value pairs inside curly braces.
- 9 An _____ is a distinct datatype in programming with its own data properties and methods.
- 10 If an object is given, as follows:

```
var ball = {
  xPos = 0,
  yPos = 0,
  diameter = 10,
  color = Color(255,100,100)
}
```

then the console will display a value of '10' with the command `console.log(_____)`.

Activity

Add More Planets

To add more planets, we need to create multiple objects. For example, to add the Earth we can create a new object as we did earlier. Instead of creating multiple objects, we can create an array of objects.

Array of Objects: Just like a normal data array, we can create an array of objects. Each object in an array is identified by an index number that represents its position in the array. In arrays, the first element index in the array is [0], the second element index is [1], and so on.

We can create an array by adding objects in place of data, separated by commas. In this activity, we can create an array of objects for planets.

```
planets = [{
  imagePath: "mars.jpg",
  name: "Mars",
  dia: "6,787 km"
},{
  imagePath: "earth.jpg",
  name: "Earth",
  dia: "12,760 km"
}];
```

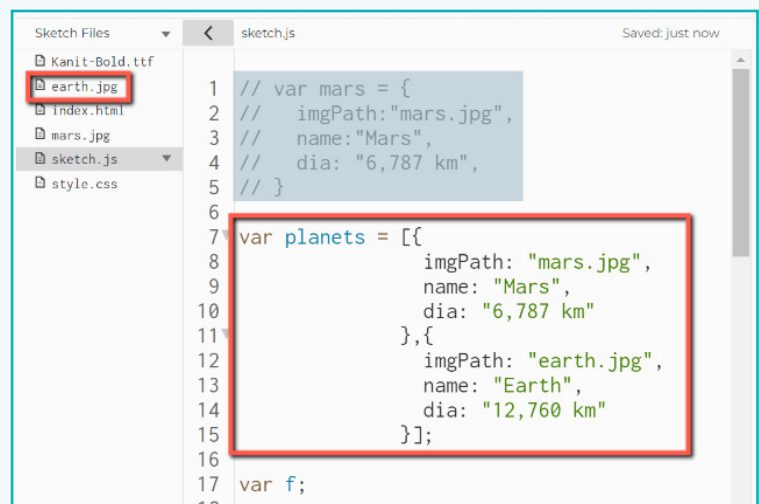
//1st Object element at index 0 for Mars

//2nd Object element at index 1 for Earth

To access a specific property of an object, we can first get an object from the array using the 'index' and then use the 'dot (.)' notation. For example, 'planets[1].name' will have value as 'Earth'. We can check other properties using the console. Let us implement this to add more planets.

1 Add multiple planets

- Add an 'Earth' texture image from the resources in the Sketch Files.
- Create an array of objects called 'planets' with properties such as 'imagePath', 'name', and 'dia'. Use values from the table.



Activity

Note that we have added a property that stores the texture image in the 'preload' function. As we have multiple elements in the array, we will use a 'for' loop to iterate through all the objects in the array and add the 'img' property using the 'loadImage()' command and 'image path'.

2 Adding texture images for all objects in the array

- Comment out the code to add a texture image for Mars in the 'preload()' function.
- Add a 'for' loop with counter 'i' starting from '0'; with condition 'i' less than the length of the planet's array element and incrementing the loop by '1' with each iteration.
- Add code to create the new 'img' property for each object planets[i] in an array and assign the texture image using the 'loadImage()' command with image path - 'planets[i].imgPath' for each planet.



```
> sketch.js Saved: 25 seconds ago
19 function preload(){
20
21   for(let i = 0; i < planets.length; i++){
22     planets[i].img = loadImage(planets[i].imgPath);
23   }
24
25   // mars.img = loadImage(mars.imgPath);
26
27   f = loadFont('Kanit-Bold.ttf');
28 }
```

Now, we have data and image files that can display multiple planets. However, at a time, we can show only one planet with the proper size on the screen. In this case, we can use a key to display the next planet and its information in the same position as a carousel.

3 Create counter for planet to be displayed

- Declare a variable 'count' at the beginning and initialize it to '0' in the setup.

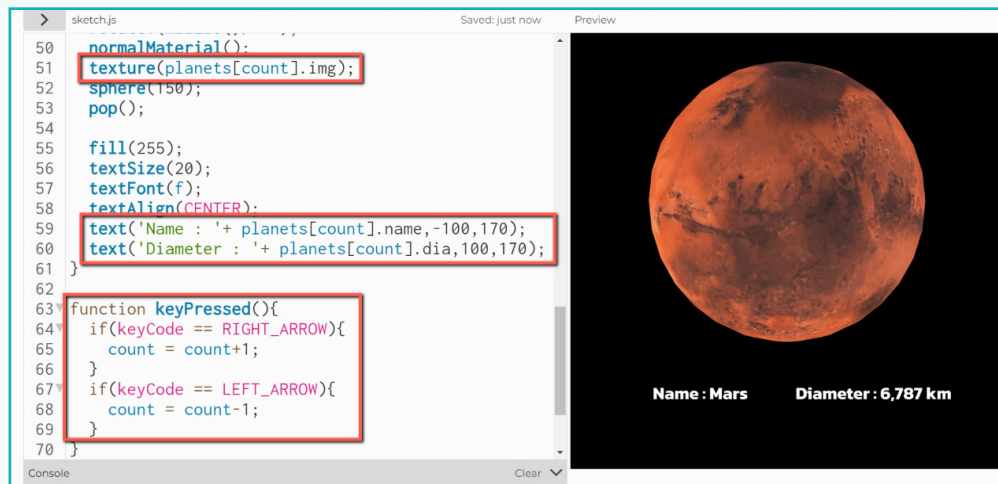


```
17 var f;
18 var count;
19
20 function preload(){
21
22   for(let i = 0; i < planets.length; i++){
23     planets[i].img = loadImage(planets[i].imgPath);
24   }
25
26   // mars.img = loadImage(mars.imgPath);
27
28   f = loadFont('Kanit-Bold.ttf');
29 }
30
31 function setup() {
32   createCanvas(500, 500, WEBGL);
33   angleMode(DEGREES);
34
35   count = 0;
36 }
```

Activity

4 Add functionality to change planet

- Update the object as 'planets[count]' from 'Mars' throughout the code i.e., in texture and text commands. Now, based on the value of the 'count' variable, the planet from the array will be displayed.
- Add the function 'keyPressed()' to detect the key press event.
- Add the 'if' conditions to detect if the default variable keycode is RIGHT_ARROW or LEFT_ARROW. These conditions will detect which key is pressed and increment or decrement the 'count' variable, respectively.



We can change the planet displayed on the screen by pressing the left or right arrow keys. We have added only two planets to our array so far (present at index locations '0' and '1'). Therefore, if the 'count' is 'less than 0' or 'greater than 1', then an error will be displayed.

To address this issue, we can add conditions so that the 'count' value will be updated cyclically and always be equal to the index values available in the array. This will create a carousel like functionality.

5 Circular switching through array

- Add the 'if ()' statement to check the condition 'count < 0'. If the condition is 'true', set the 'count' value as the index of the last object in the array i.e., (planets.length - 1). This way, we will have a cyclic counter. If the first planet is displayed and the 'left arrow' key is pressed, the last planet will be displayed on the array list.
- Below the 'if()' statement, add the 'else if()' statement to check the condition 'count == planets.length' i.e., if the counter moves to the next value from the index of the last index available in the array. Since the index of the last object element in an array is always (planets.length - 1) and no array object will be available at the index (planets.length), we can go to the first planet from the last one.
- Therefore, if the condition is 'true', set the 'count' value to '0'.

Activity



With this code, the entire application is ready. We can now use arrow keys to go through the planets shown in 3D with their details. This type of interactive 3D application is the future of Digital Encyclopedia.

Practice Activity: Update the code to include more planets and celestial objects with their detailed information data.

In this lesson, we have learnt and implemented the concepts of Object-Oriented Programming to build a digital encyclopedia having 3D models and interactivity. We can build many more such interactive encyclopedia apps that can help us and even elders to learn things in a more engaging way.

Exercise

Answer the questions with one sentence.

- 11 What will be the output of `console.log(planets.length - 1)`, if an array of objects 'planets' is updated to have objects for all the eight planets given in the table with five properties each?



- 12 Write the code that returns value as "Venus", if the 'key:value' pair for the second property is name: "Venus" of the second object in the 'planets' array.



Well done, Zian! I'm really impressed with the interactive 3D planets wiki you've created using object-oriented programming. You've come a long way in just one lesson.

Thank you, Techna! It was a lot of fun working with objects and arrays. I never thought I could create my own digital encyclopedia with computer programming.



Tech Flash

- **WEBGL** : WebGL is a library for 3D rendering mode available in p5js that is used to display 3D shapes and objects.
- **Texture** : Texture is the visual characteristic of the surface that wraps around a 3D geometry.
- **Object-Oriented Programming** : Object-Oriented Programming is a programming paradigm that organizes code based on the concept of objects, which are entities that encapsulate data as well as methods.
- **Object** : Object is a unique data entity that can contain properties and methods.
- **Object Literal** : Object Literal is a method to create objects independently and is a comma-separated list of name-value pairs inside of curly braces.
- **Array of Objects** : Array of Objects is an array containing its elements as objects where each object in an array is identified by an index number in the array.